



Nobody Gets Promoted for Simplicity

"Simplicity is a great virtue, but it requires hard work to achieve and education to appreciate. And to make matters worse, complexity sells better." – Edsger Dijkstra

I think there's something quietly screwing up a lot of engineering teams. In interviews, in promotion packets, in design reviews: the engineer who overbuilds gets a compelling narrative, but the one who ships the simplest thing that works gets... nothing.

This isn't intentional, of course. Nobody sits down and says, "*let's make sure the people who over-engineer things get promoted!*" But that's what can happen (and it has been, over and over again) when companies evaluate work incorrectly.

Picture two engineers on the same team. Engineer A gets assigned a feature. She looks at the problem, considers a few options, and picks the simplest. A straightforward implementation, maybe 50 lines of code. Easy to read, easy to test, easy for the next person to pick up. It works. She ships it in a couple of days and moves on.

Engineer B gets a similar feature. He also looks at the problem, but he sees an opportunity to build something more "robust." He introduces a new abstraction layer, creates a pub/sub system for communication between components, adds a configuration framework so the feature is "extensible" for future use cases. It takes three weeks. There are multiple PRs. Lots of excited emojis when he shares the document explaining all of this.

Now, promotion time comes around. Engineer B's work practically writes itself into a promotion packet: "*Designed and implemented a scalable event-driven architecture, introduced a reusable abstraction layer adopted by multiple teams, and built a configuration framework enabling future extensibility.*" That practically screams Staff+.

But for Engineer A's work, there's almost nothing to say. "*Implemented feature X.*" Three words. Her work was better. But it's invisible because of how simple she made it look. You can't write a compelling narrative about the thing you *didn't* build. **Nobody gets promoted for the complexity they avoided.**

Complexity looks smart. Not because it is, but because our systems are set up to reward it. And the incentive problem doesn't start at promotion time. It starts before you even get the job.

Think about interviews. You're in a system design round, and you propose a simple solution. A single database, a straightforward API, maybe a caching layer. The interviewer is like: "*What about scalability? What if you have ten million users?*" So you add services. You add queues. You add sharding. You draw more boxes on the whiteboard. The interviewer finally seems satisfied now.

What you just learned is that complexity impresses people. The simple answer wasn't wrong. It just wasn't interesting enough. And you might carry that lesson with you into your career. To be fair, interviewers sometimes have good reasons to push on scale; they want to see how you think under pressure and whether you understand distributed systems. But when the takeaway for the candidate is "simple wasn't enough," something's off.

It also shows up in design reviews. An engineer proposes a clean, simple approach and gets hit with *"shouldn't we future-proof this?"* So they go back and add layers they don't need yet, abstractions for problems that might never materialize, flexibility for requirements nobody has asked for. Not because the problem demanded it, but because the room expected it.

I've seen engineers (and have been one myself) create abstractions to avoid duplicating a few lines of code, only to end up with something far harder to understand and maintain than the duplication ever was. Every time, it felt like the right thing to do. The code looked more "professional." More engineered. But the users didn't get their feature any faster, and the next engineer to touch it had to spend half a day understanding the abstraction before they could make any changes.

Now, let me be clear: complexity is sometimes the right call. If you're processing millions of transactions, you might need distributed systems. If you have 10 teams working on the same product, you probably need service boundaries. When the problem is complex, the solution (probably) should be too!

The issue isn't complexity itself. It's unearned complexity. There's a difference between *"we're hitting database limits and need to shard"* and *"we might hit database limits in three years, so let's shard now."*

Some engineers understand this. And when you look at their code (and architecture), you think *"well, yeah, of course."* There's no magic, no cleverness, nothing that makes you feel stupid for not understanding it. And that's exactly the point.

The actual path to seniority isn't learning more tools and patterns, but learning when not to use them. **Anyone can add complexity. It takes experience and confidence to leave it out.**

So what do we actually do about this? Because saying “keep it simple” is easy. Changing incentive structures is harder.

If you’re an engineer, learn that simplicity needs to be made visible. The work doesn’t speak for itself; not because it’s not good, but because most systems aren’t designed to hear it.

Start with how you talk about your own work. “Implemented feature X” doesn’t mean much. But *“evaluated three approaches including an event-driven architecture and a custom abstraction layer, determined that a straightforward implementation met all current and projected requirements, and shipped in two days with zero incidents over six months”*, that’s the same simple work, just described in a way that captures the judgment behind it. The decision *not* to build something is a decision, an important one! Document it accordingly.

In design reviews, when someone asks “shouldn’t we future-proof this?”, don’t just cave and go add layers. Try: *“Here’s what it would take to add that later if we need it, and here’s what it costs us to add it now. I think we wait.”* You’re not pushing back, but showing you’ve done your homework. You considered the complexity and chose not to take it on.

And yes, bring this up with your manager. Something like: *“I want to make sure the way I document my work reflects the decisions I’m making, not just the code I’m writing. Can we talk about how to frame that for my next review?”* Most managers will appreciate this because you’re making their job easier. You’re giving them language they can use to advocate for you.

Now, if you do all of this and your team still only promotes the person who builds the most elaborate system... that’s useful information too. It tells you something about where you work. Some cultures genuinely value simplicity. Others say they do, but reward the opposite. If you’re in the second kind, you can either play the game or find a place where good judgment is actually recognized. But at least you’ll know which one you’re in.

If you’re an engineering leader, this one’s on you more than anyone else. You set the incentives, whether you realize it or not. And the problem is that most promotion criteria are basically designed to reward complexity,

even when they don't intend to. "Impact" gets measured by the size and scope of what someone built, which more often than not matters! But what they avoided should also matter.

So start by changing the questions you ask. In design reviews, instead of "have we thought about scale?", try *"what's the simplest version we could ship, and what specific signals would tell us we need something more complex?"* That one question changes the game: it makes simplicity the default and puts the burden of proof on complexity, not the other way around!

In promotion discussions, push back when someone's packet is basically a list of impressive-sounding systems. Ask: *"Was all of that necessary? Did we actually need a pub/sub system here, or did it just look good on paper?"* And when an engineer on your team ships something clean and simple, help them write the narrative. "Evaluated multiple approaches and chose the simplest one that solved the problem" is a compelling promotion case, but only if you actually treat it like one.

One more thing: pay attention to what you celebrate publicly. If every shout-out in your team channel is for the big, complex project, that's what people will optimize for. Start recognizing the engineer who deleted code. The one who said "we don't need this yet" and was right.

At the end of the day, if we keep rewarding complexity and ignoring simplicity, we shouldn't be surprised when that's exactly what we get. But the fix isn't complicated. Which, I guess, is kind of the point.

opinion

career, culture, engineering, leadership, teams

Discover more from Terrible Software

Subscribe to get the latest posts sent to your email.

More Like This



[What Actually Makes You Senior](#)

November 25, 2025



[Why Engineers Hate Their Managers \(And What to Do About It\)](#)

June 24, 2025



[Why Your 'Harmonious' Team Is Actually Failing](#)

March 12, 2025

[Bluesky](#), [GitHub](#), [LinkedIn](#), [X](#)